

① Approach to Solve Low Level Design Problems in an Interview :-

1. Identify the Requirements.
2. Model the Problems (Try to identify different classes that will be used in that problem).

3. Establish the classes and their relationships.

4. Use SOLID Principles and Design Patterns.

5. Write the Code.

① SOLID Principles :-

1. Single Responsibility Principle

(SRP) :- If a class handles multiple responsibilities, break it into smaller classes, each having only one responsibility.

class Report

```
{ public void generateReport()
```

```
{  
    S.O.P("Generate Report");  
}
```

```
public void saveToFile()
```

```
{  
    S.O.P("Saving to File");  
}
```

```
}
```

class ReportGenerator // SRP applied

```
{  
    public void generateReport()  
    {  
        S.O.P( "Generate Report" );  
    }  
}
```

class ReportSaver

```
{  
    public void saveToFile()  
    {  
        S.O.P( "Saving To File" );  
    }  
}
```

2. Open / Closed Principle (O) :- Modifying existing code for new feature increases the chance of bugs. So in order to avoid that, use abstraction and polymorphism to extend functionality without modifying the existing code. [So code is open for extension but closed for modification].

```
class InvoicePrinter
```

```
{ public void printInvoice(String type)
```

```
{ if (type.equals ("PDF"))
```

```
{ S.O.P (" PDF");
```

```
}
```

```
else if (type.equals ("HTML")
```

```
S.O.P (" HTML");
```

```
}
```

```
}
```

① interface Invoice // ocp Applied

```
{  
    void print();  
}
```

« class POF Invoice implements Invoice

```
{  
    public void print()  
    {  
        S.O.P("POF");  
    }  
}
```

```
class HTML Invoice implement Invoice  
{  
    public void print()  
    {  
        S.O.P("HTML");  
    }  
}
```


3. Liskov Substitution Principle (LSP):-

Subclasses behave differently than their base class, leading to unexpected results. To fix that issue, ensure subclasses honor the contract of the base class. So whenever extending a base class, always think that if subclass will be usable anywhere the parent class is used.

```
class Bird
```

```
{
```

```
    p.v fly()
```

```
    {
```

```
        s.o.p ("Bird flying");
```

```
    }
```

```
}
```

```
class Ostrich extends Bird
```

```
{
```

```
    p.v fly()
```

```
    { throw new Exception("Ostrich cant  
fly");
```

```
    }
```

```
}
```

$$\textcircled{n} = \textcircled{y} = \sum_i^{\downarrow} i$$

const

interface Bird() // LSP Applied

```
{  
    void eat();  
}
```

interface FlyingBird extends Bird

```
{  
    void fly();  
}
```

class Sparrow implements FlyingBirds

```
{  
    public void eat();  
    public void fly();  
}
```

class Ostrich implements Bird

{

public void eat()

{

}

}

4. Interface Segregation Principle

(ISP) :- Sometimes a class is forced to implement methods it doesn't need. To fix that issue, split larger interface into smaller ones.

eg:-

```
interface worker  
{  
    void work();  
    void eat();  
}
```

```
class Robot implements worker  
{  
    public void work();  
    public void eat()  
    { throw new Exception();  
    }  
}
```

```
interface Workable
```

// ISP Applied

```
{  
    void work();  
}
```

```
}
```

```
interface Eatable
```

```
{  
    void eat();  
}
```

```
}
```

class Robot implements
 Workable

```
{  
    p.v.work();  
}
```

class Human implements Workable, Eatable

```
{  
    p.v.work();    p.v.eat();  
}
```

```
}
```

5. Dependency Inversion Principle (DIP) :-

It states that high level module should not depend on low level module but both should depend on abstractions. Abstraction

should not depend on details rather details should depend on abstraction.

eg :-

```
class LightBulb
```

```
{ public void turnOn() {}
```

```
    p.v turnOff() {}
```

```
}
```

```
class Switch
```

```
{ Lightbulb bulb = new LightBulb(),
```

```
    p.v . operate()
```

```
    { bulb.turnOn(),
```

```
    }
```

⑥ interface Switchable // DIP applied
{
 void turnOn();
 void turnOff();
}

class LightBulb implements Switchable

{
 p.v turnOn();
 p.v turnOff();
}

```
class Switch
```

```
{ Switchable device;
```

```
public Switch (Switchable device)
```

```
{ this.device = device;
```

```
}
```

```
public void operate()
```

```
{ device.turnOn();
```

```
}
```

```
}
```